PHT Literature Review using Large Language Models

A Four-Model Ensemble for Framework Coding of Accessibility Research

Pavan Prasad Gorintla
Smirthya Somaskantha Iyer

Northern Arizona University
School of Informatics, Computing, and Cyber Systems
CS 599: Contemporary Developments in Applications of Large Language Models

May 5, 2026

Abstract

Coding 215 papers from three decades of accessibility research against an eight-dimension design framework took two trained reviewers six months of work. We built a retrieval-augmented language model pipeline that runs the same coding task in parallel across four large language models from four different providers, and compared the model outputs against the human consensus. The system applies prompt engineering techniques covered in this course (four-part prompt structure, role prompting, chain-of-thought reasoning, calibrated confidence anchors, and dimension-specific few-shot examples) and exposes the results through a web interface that shows model-human agreement at a glance. We report on the design of the system, the prompt rebuild that produced our biggest quality gains, and the agreement patterns across the four models. The single most useful lesson from this project: a smaller model with a well-engineered prompt outperforms a larger model with a weak one.

1. Motivation

Researchers in our lab spent six months hand-coding 215 accessibility papers from the ACM ASSETS proceedings (1994 to 2023) against an eight-dimension framework on Playful Health Technology (PHT) [3]. Two trained reviewers each coded every paper independently and met to reconcile disagreements. The slow part was not reading the papers. It was the conversations needed to settle what should sit at a 2 versus a 3 on dimensions where the framework leaves room for interpretation.

The labor cost of systematic literature review is well documented in the broader research-automation literature, where recent surveys have cataloged how large language models are increasingly used to compress this work [7, 9, 10]. Our project sits in that space. We are testing whether language models can produce coding

decisions that align with trained human reviewers on a real corpus and a real framework, and asking what the disagreement cases reveal about both the models and the framework.

Multiply 215 papers by 8 dimensions and you get more than 1,700 individual coding decisions per reviewer. The work scales linearly with both corpus size and framework breadth. Add a second corpus or revise the framework, and the cost doubles or worse. This is the bottleneck for systematic reviews in our research area, not just for our lab but for any group running a framework analysis at this scale.

Large language models can read structured prompts and return structured outputs at a rate that humans cannot match. If a language model can produce coding decisions that align with trained human reviewers most of the time, it could compress months of work into hours, with humans focusing their effort on the cases where models disagree. That is the bet this project tests. We do not claim that language models replace human coders. We claim that comparing model output against human output reveals where models are reliable, where they fail, and what those failure modes can teach us about both the framework and the underlying technology.

2. Objective and Scope

The project has one primary objective and three subordinate goals.

Primary objective. Build and evaluate a retrieval-augmented system that uses four different large language models to code the 215-paper PHT corpus against the same eight-dimension framework that two human reviewers used. Surface the results in an interface that lets a researcher see model-human agreement and inspect the evidence each model used.

Subordinate goals.

- Apply the prompt engineering techniques covered in this course (four-part prompt structure, role prompting, chain-of-thought, few-shot in-context learning, calibrated confidence) and measure the quality lift from each.
- Use four models from different providers (one commercial flagship, one commercial speed-optimized, one large open-weight, one mid-sized open-weight) so that inter-model agreement carries a useful signal.
- Build a web interface that supports the human-in-the-loop workflow, where a researcher can re-code a paper, view the chain of reasoning each model produced, and see the cited evidence chunks.

Requirements. The system must: (1) accept any paper from the ASSETS corpus as a PDF and return eight dimension scores per model; (2) display model-human agreement in a single heatmap view; (3) cite the specific text passages each model relied on; (4) allow a researcher to manually re-code any cell and see the impact immediately; and (5) produce output in a machine-readable format suitable for downstream statistical analysis.

Out of scope. Fine-tuning was considered and ruled out for this iteration. The course material is explicit

that fine-tuning is not always preferable to a strong prompt on a strong base model, and our timeline favored prompting work over a multi-week training pipeline. We also did not modify the wording of the eight rubric questions themselves, since those belong to the lab's instrument and changing them would have invalidated comparison with the existing human consensus data.

3. Methods, Tools, and Datasets

3.1 Dataset

The corpus is 215 papers from the ACM ASSETS conference, spanning 1994 through 2023. ASSETS is the leading peer-reviewed venue for accessibility research, which makes the corpus a credible cross-section of three decades of work in the area. Each paper enters the pipeline as a PDF, and a parallel set of human consensus codings exists for 182 of the 215 papers from prior lab work. Those 182 codings serve as the ground truth against which we evaluate model outputs.

We chose ASSETS because the existing human consensus codings make it possible to evaluate model outputs against a real, laboriously produced baseline rather than against a synthetic or crowd-sourced one. The quality of the evaluation is only as good as the ground truth, and six months of trained-reviewer work is credible ground truth.

3.2 The PHT Framework

The framework, developed by Duval [3] through a participatory research-through-design methodology with multiple disability populations, places each paper on eight design spectra, each scored 1 to 5. The eight dimensions are: Fun versus Utility (design intent: entertainment versus clinical), Play versus Game (open exploration versus rule-bound structure), Skill versus Chance (primary outcome driver), Solo versus Social (player count and interaction), Sequential versus Simultaneous (turn rhythm), Synchronous versus Asynchronous (timing of participation), Competitive versus Collaborative (against or with others), and Symmetric versus Asymmetric (role uniformity across players). Each dimension is scored on a 1-to-5 scale where 1 and 5 represent the poles and 3 represents a balanced midpoint.

The questions are written for trained human coders. They contain instructional preambles and parenthetical examples that were a source of misclassification in our early prompts, an issue we address in Section 4.3. We used Duval's original question wording without modification to preserve comparability with the existing human consensus data.

3.3 Models

We selected four large language models, one from each of four different providers, to ensure that inter-model agreement is maximally informative.

- **GPT-5.1** (OpenAI). A commercial closed-source flagship, used as our quality reference.
- **Gemini 2.5 Flash** (Google). A commercial closed model optimized for inference speed, included to test whether a faster, cheaper model can keep pace with a flagship on this task.
- **Llama 3.3 70B** (Meta, served via Groq). A large open-weight model running on specialized hardware, representing the open-source tier at scale.
- **Mistral Small 3.1** (Mistral AI). A European open-weight model from a different lab with different training data and architecture decisions.

We chose four providers deliberately. When all four models agree on an answer, that agreement carries useful signal precisely because the models do not share training data, architectures, or alignment objectives. When all four disagree with each other on the same paper, that pattern points to ambiguity in either the paper or the framework itself. When all four agree but disagree with the human consensus, that is the most telling case of all: it reveals systematic blind spots that no single-model evaluation would have surfaced. This kind of cross-model comparison sits in a growing line of work on using LLMs as evaluators or annotators in NLP tasks [11, 6].

3.4 Tools and Infrastructure

The backend is built on FastAPI with server-sent events for live coding updates. Embeddings are computed using MiniLM-L6-v2, a lightweight sentence transformer that offers a favorable balance between embedding quality and inference speed. Vector indices are managed with FAISS, with embeddings cached in SQLite so each paper is embedded only once. The frontend is vanilla JavaScript with no build step, keeping the system easy to deploy and inspect. Outputs are stored as JSONL on disk rather than in a database, making them easy to version, back up, and process with standard command-line tools.

4. Solution Design

4.1 Pipeline Architecture

The system follows a retrieval-augmented generation (RAG) architecture, which the course covers as the standard remedy for reducing hallucination in document-grounded tasks. The pipeline proceeds through six stages.

1. **Ingest.** A paper enters as a PDF. The chunker produces `TextChunk` objects, each carrying its page range and character offsets so specific passages can be cited in the UI.
2. **Embed.** Chunks are embedded into vector representations and stored per-paper in a FAISS index. Embeddings are cached in SQLite so a paper is embedded only once.

3. **Retrieve.** For each of the eight rubric questions, a `HybridRetriever` runs both lexical search and dense vector search and combines the top-K results from each. For papers under 30 chunks, retrieval is skipped entirely and the full paper is passed, avoiding evidence loss on short papers where retrieval would be wasteful.
4. **Prompt.** Retrieved passages are inserted into a four-part prompt (described in Section 4.3) and sent in parallel to all four models.
5. **Parse and store.** Each model returns strict JSON. The pipeline normalizes the answer (1 to 5, `Unknown`, or `Not Applicable`), records the cited evidence chunks, the model's reasoning text, its self-reported confidence, and a `decision_mode` flag indicating which retrieval path produced the answer. Records are written as JSONL.
6. **Display.** The web frontend renders a per-paper heatmap. Cells are color-coded by answer (1 to 5 on a viridis ramp). When human consensus data exists for a paper, cells receive a green border for matches and a red border for mismatches. A short tag on each cell indicates how the model arrived at its answer, and a red exclamation mark flags cells where confidence dropped below 0.65.

4.2 Why RAG, Not Full-Paper Context

A naive approach would send every paper in full to every model for every dimension. This fails for two reasons. First, papers range from 4 to 30+ pages, with several exceeding the context windows of the open-weight models even at 70B scale. Second, even when a paper fits, sending the entire text for a question about (say) turn-rhythm forces the model to do its own implicit retrieval, and we lose visibility into which passages the model actually relied on. Hybrid retrieval scopes the model's attention to the passages that matter, improving both answer quality and evidence traceability.

The 30-chunk threshold for switching to full-paper context is a pragmatic compromise. Below that threshold, the cost of retrieval outweighs the benefit. Above it, a retrieval pass is essential.

4.3 The Prompt Rebuild

This is where the project's biggest quality lift came from, and where the course material had the most direct impact.

The first version of our prompt (v1) was zero-shot. It told the model it was a "careful research assistant," gave it the rubric question, and dumped the retrieved chunks. It forced the model to pick a 1 to 5, or to say `Unknown`, with reasoning produced after the answer. There was no role beyond "research assistant," no framework definition, no examples, and no calibration. Inter-model agreement was low, and confidence scores returned by the models were inflated, often above 0.8 even on demonstrably wrong answers. The pipeline included a forced-choice fallback: when the first pass returned `Unknown`, a second prompt told the model to pick something between 1 and 5 anyway. Analysis of those forced answers showed they correlated with model-human disagreement, confirming the lecture's warning that forcing a model to answer when

evidence is absent manufactures the exact hallucination pattern the system is supposed to avoid. Recent work meta-evaluating local LLMs on subjective scoring tasks has documented similar patterns [4].

We rewrote the prompt from scratch using techniques the course covered in lectures 14 through 22. The new prompt (v2) is approximately six times longer than v1 and applies seven changes.

1. **Four-part structure.** Explicit sections for instruction, framework context, the rubric question, and the retrieved paper context, following the prompt-design pattern from the course.
2. **Role prompting.** The model is cast as a "trained coder for a systematic literature review on Playful Health Technology," giving it a more specific identity than the generic "research assistant" used in v1.
3. **Framework context block.** Every prompt includes a paragraph defining what PHT is and what each dimension measures, so the model is not inferring meaning from parenthetical examples in the rubric question.
4. **Chain-of-thought reasoning.** The output schema requires reasoning *before* the answer, forcing the model to commit to a line of thinking rather than rationalizing a snap judgment after the fact.
5. **Dimension-specific few-shot examples.** One worked example per dimension, anchored on a paper where all four models and the human consensus agreed in our pilot run. Few-shot prompting is a standard course-covered technique for tasks where the reasoning pattern differs by category.
6. **Calibrated confidence anchors.** The prompt now defines what 0.3, 0.6, and 0.9 mean on the confidence scale, so scores are comparable across models and dimensions rather than being a free-floating self-rating.
7. **Honest abstention.** The model is allowed to answer `Unknown` when evidence is absent, or `Not Applicable` when the paper does not describe a play system at all. The forced-choice fallback was removed.

The output schema was also reordered. v1 asked for `answer` first, then `reasoning`; v2 asks for `reasoning` first, then `evidence`, then `answer`, then `confidence`. This ordering has an outsized effect: producing the reasoning first nudges the model into a chain-of-thought pattern, since the answer field is the last thing the model writes and is therefore conditioned on the reasoning that came before it.

4.4 The User Interface

The frontend has three views: a corpus browser, a per-paper heatmap, and a detail panel. The detail panel opens when a user clicks any cell and shows the model's reasoning, the chunks it cited as evidence, and all chunks the retriever returned, which lets a human researcher see whether retrieval missed something the model needed. The interface streams live updates over server-sent events so that when a user re-codes a paper, the heatmap fills in cell-by-cell as the four models return their answers.

5. Findings and Results

5.1 Where the Four Models Agree

The dimensions where all four models agreed with the human consensus most often were those whose answers tend to be stated explicitly in the papers themselves. **Play vs. Game** (rule structure) was the easiest dimension: papers that describe a system with rules typically say so directly in the methods section, and all four models picked up the explicit signal reliably. Four-model agreement matching human consensus reached 76% on this dimension.

Other strong dimensions: **Solo vs. Social** at 71%, where player count is typically described in methods sections; **Symmetric vs. Asymmetric** at 68%, where role differences are usually described when they exist; and **Skill vs. Chance** at 64%, where outcome drivers are typically mentioned concretely in game descriptions.

5.2 Where the Four Models Fail

The dimensions where models struggled were those requiring inference from indirect cues rather than extraction from explicit statements. **Sync vs. Async** had the lowest agreement in the corpus at 38%. Most papers do not discuss the timing of participation directly, so models inferred it from descriptions of the user study or the system's interaction model, and they inferred differently than human reviewers did.

Fun vs. Utility produced 42% agreement, as design intent often requires interpretation beyond what is explicitly stated. **Compete vs. Collab** reached 45%, with surprising failures on systems where players compete against their own previous scores. Self-competition is a recognized edge case in the framework that even human coders disagree on, and the models showed the same pattern.

5.3 The Most Useful Failure Mode: Same-Direction Errors

The most analytically valuable result was not where models did well or poorly individually, but where all four models failed in the same direction. On the sample paper shown in the heatmap interface, all four models incorrectly coded **Compete vs. Collab** as 5 (fully collaborative) while the human consensus was 1 (fully competitive). When four models trained on different data and built with different objectives all make the same mistake, the model is not the problem. Either the paper's text leaves real interpretive room, or the framework's wording for that dimension is ambiguous in a way that nudges multiple models toward the wrong answer.

This pattern, using inter-model agreement as a diagnostic for framework or paper ambiguity rather than purely as a measure of model quality, is one of the genuinely novel contributions of the project. It would not be visible from a single-model evaluation.

5.4 Prompt v1 versus v2

The most concrete demonstration of the prompt rebuild's impact came from re-coding a fixed sample of papers with v1 and then re-coding them with the v2 chain-of-thought, few-shot prompt on the same models. v2 produced higher model-human agreement across every dimension tested, and the gap was largest on the inference-heavy dimensions where v1 produced the most forced, ungrounded guesses. Critically, smaller models running v2 outperformed larger models running v1, matching the course's framing that prompting is the highest-leverage tool available before considering fine-tuning.

5.5 Decision Modes

A useful side-effect of removing the forced-choice fallback is that the pipeline now records how a model arrived at each answer. The decision modes are: `full_context` (small paper, no retrieval needed), `retrieval` (standard retrieval pass produced an answer), `expanded_retry` (first pass returned `Unknown`, wider retrieval resolved it), `unresolved` (`Unknown` remained even after expansion), and `not_applicable` (paper does not describe a play system). The honesty cost is real: more `Unknown` answers than the v1 system produced. The honesty benefit is also real: when the system returns `Unknown`, the answer is genuinely uncertain rather than a forced guess masquerading as a confident 1-to-5.

6. Limitations

We want to be direct about what this project does not establish.

Agreement is a proxy for correctness, not correctness itself. Two trained human reviewers can disagree on the same paper, which is exactly why the lab's coding process required reconciliation. When we report model-human agreement, we are reporting agreement with the consensus that emerged from that reconciliation process, which is itself an interpretive product. A model that produces a 4 on a paper where the consensus was 3 might be wrong, or might have caught something the human reviewers missed. The data does not let us distinguish those cases. The broader human-evaluation literature in NLP makes this same point [8, 2].

The corpus is one corpus, the framework is one framework. We tested on ASSETS papers and the PHT framework. We have no evidence that the approach generalizes to a different conference, a different framework, or a different research domain. Generalization would require running the full pipeline against a second corpus, which was outside our timeline.

Closed models are unauditable. GPT-5.1 and Gemini 2.5 Flash are commercial models we cannot inspect. We cannot rule out that their training data contains some subset of the ASSETS corpus, which would inflate agreement scores in ways we cannot detect. The two open-weight models (Llama and Mistral) provide a useful check because their training data is more transparent, but the closed models remain a known unknown. Detecting whether specific documents were part of a model's training data is itself an open research problem [5], which means we cannot easily test for contamination directly.

Retrieval quality caps everything downstream. If the retriever returns the wrong chunks, no amount of prompt engineering will recover the right answer. We did not invest in cross-encoder re-ranking or query expansion in this iteration, both of which are standard improvements for advanced RAG. A paper whose relevant evidence sits in a passage the retriever missed will be coded incorrectly regardless of which model handles it.

Free API rate limits forced serial processing. We could not run the full corpus across all four models in parallel without hitting rate limits, particularly on the Mistral and Gemini free tiers. Production deployment would require paid API access.

6.1 Project Outcomes vs. Original Goals

Two design decisions in the final system differ from the original proposal, and both are worth explaining. The proposal planned to use AI Verde, a unified LLM gateway, to dispatch all models through a single API endpoint, with a six-model ensemble and temperature sensitivity experiments to identify optimal inference settings per model. In practice, we moved away from this design for two reasons. First, as we built the pipeline we found that routing all models through a single unified API removed the ability to set provider-specific parameters, handle provider-specific error modes gracefully, and inspect raw responses at the provider level. Connecting to each provider's API individually gave us finer control over retry logic, timeout handling, and the JSON parsing step that turns each model's raw text into a structured coding record. Second, temperature sensitivity analysis became less relevant once the v2 prompt introduced calibrated confidence anchors and chain-of-thought reasoning: the structured output format constrained variability in ways that made temperature sweeps redundant for our purposes. We reduced the ensemble from six models to four, dropping two planned open-weight models, and replaced the unified-API approach with direct per-provider integration. These tradeoffs favored reliability and interpretability of the pipeline over the breadth of the original plan.

The full codebase is publicly available at https://github.com/gorintlapavanprasad/PHT_Literature_review. The repository is organized into three directories: `paper_annotation` (the RAG pipeline and prompt templates), `pht_ui` (the FastAPI backend and frontend heatmap interface), and `reference code` (earlier prototype versions). The system is implemented primarily in Python (83%) with HTML for the frontend (17%).

The objective was to build a working system that codes the corpus across four models and surfaces the results against human consensus, applying course-appropriate prompt engineering. That objective was met: the system runs, the prompts apply the techniques the course covers, and the UI surfaces both the answers and the reasoning. All five stated requirements are addressed. The harder, longer-term question, whether language models can reliably substitute for human coders on framework-coding tasks at scale, is one the project can begin to answer but cannot settle. Future work, in rough order of payoff: add cross-encoder re-ranking on retrieved chunks; implement self-consistency by sampling each question three times and taking a majority vote; fine-tune one open-weight model on the 182-paper consensus dataset; and test the entire pipeline against a second framework on a second corpus.

7. Workload Division

Pavan Prasad Gorintla. Designed and implemented the retrieval-augmented pipeline (chunking, embedding, hybrid retrieval, four-provider integration). Wrote the v1 and v2 prompts, including the framework context block, calibrated confidence anchors, and the eight dimension-specific few-shot examples. Built the FastAPI backend, the JSONL persistence layer, and the web frontend (heatmap, evidence panel, corpus browser, human-consensus comparison feature). Removed the forced-choice fallback and added the `not_applicable` decision mode.

Smirthya Somaskantha Iyer. Designed the four-model ensemble framework and the cross-model comparison methodology. Implemented the inter-rater reliability metrics and the dimension-by-dimension agreement analysis. Characterized the failure modes, including the same-direction error pattern and the cross-model bias analysis. Produced the validation analysis comparing v1 against v2 prompt outputs on a fixed held-out sample.

8. Conclusion

The single sentence that captures what we learned: prompt engineering does most of the work that determines whether a language model produces useful framework codings, and the gap between a weak prompt and a strong one is wider than the gap between a smaller model and a larger one. The four-model ensemble makes that visible in a way a single-model evaluation cannot. The cases where all four models agree with humans tell us where the framework is unambiguous and the evidence is explicit. The cases where all four agree with each other but not with humans tell us where the framework or the paper has interpretive room that even diverse models resolve the same way. And the honest `Unknown` answers, which an earlier version of the system hid behind forced guesses, give downstream researchers an accurate picture of where the corpus has gaps the system cannot fill. The position we land on is consistent with recent qualitative-research work that frames AI as a co-researcher rather than a replacement: useful when a human stays in the loop to make the judgment calls the framework cannot encode [1].

References

[1] Costa, A. P., Bryda, G., Christou, P. A., & Kasperiuniene, J. (2025). AI as a Co-researcher in the Qualitative Research Workflow: Transforming Human-AI Collaboration. *International Journal of Qualitative Methods*, 24, 16094069251383739. doi:10.1177/16094069251383739

[2] Derczynski, L. (2016). Complementarity, F-score, and NLP Evaluation. In N. Calzolari et al. (Eds.), *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*, pp. 261–266. Portorož, Slovenia: European Language Resources Association.

[3] Duval, J. S. (2022). *Playful Health Technology: A Participatory, Research through Design Approach to*

Applications for Wellness (Doctoral dissertation). UC Santa Cruz. Retrieved from ProQuest Dissertations & Theses.

[4] Isaza-Giraldo, A., Bala, P., & Pereira, L. (2025). Meta-Evaluating Local LLMs: Rethinking Performance Metrics for Serious Games. arXiv:2504.12333. doi:10.48550/arXiv.2504.12333

[5] Maini, P., Jia, H., Papernot, N., & Dziedzic, A. LLM Dataset Inference: Did You Train on My Dataset? arXiv preprint.

[6] Nasution, A. H., & Onan, A. (2024). ChatGPT Label: Comparing the Quality of Human-Generated and LLM-Generated Annotations in Low-Resource Language NLP Tasks. *IEEE Access*, 12, pp. 71876–71900. doi:10.1109/ACCESS.2024.3402809

[7] Scherbakov, D., Hubig, N., Jansari, V., Bakumenko, A., & Lenert, L. A. (2025). The Emergence of Large Language Models (LLM) as a Tool in Literature Reviews: An LLM Automated Systematic Review. *Journal of the American Medical Informatics Association*, 32(6), pp. 1071–1086. doi:10.1093/jamia/ocaf063

[8] Schuff, H., Vanderlyn, L., Adel, H., & Vu, N. T. (2023). How to do human evaluation: A brief introduction to user studies in NLP. *Natural Language Engineering*, 29(5), pp. 1199–1222. doi:10.1017/S1351324922000535

[9] Silva, N., & Wickramaarachchi, D. (2025). Enhancing Systematic Literature Reviews: Evaluating the Performance of LLM-Based Tools Across Key Systematic Literature Review Stages. In *2025 5th International Conference on Advanced Research in Computing (ICARC)*, pp. 1–6. doi:10.1109/ICARC64760.2025.10963273

[10] Thode, L., Iftikhar, U., & Mendez, D. (2025). Exploring the use of LLMs for the selection phase in systematic literature studies. *Information and Software Technology*, 184, 107757. doi:10.1016/j.infsof.2025.107757

[11] Wang, R., Guo, J., Gao, C., Fan, G., Chong, C. Y., & Xia, X. (2025). Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proceedings of the ACM on Software Engineering*, 2(ISSTA), Article ISSTA086, pp. 1955–1977. doi:10.1145/3728963